

PRÁCTICA DE LABORATORIO N.º 1

SOCKETS TCP MULTHILO EN JAVA

Atención concurrente de múltiples clientes con ExecutorService

Nombre: Jose Manuel Gallardo Laura

1. Objetivos

Objetivo general

Construir, paso a paso, un servidor TCP capaz de atender varios clientes de forma concurrente mediante hilos, y verificar su funcionamiento real entre dos equipos conectados en red.

Objetivos específicos

- Identificar el carácter bloqueante de `accept()` y de las operaciones de lectura sobre un socket.
- Comprobar experimentalmente la limitación de un servidor secuencial (un solo hilo).
- Delegar la atención de cada cliente a un hilo independiente usando un pool (`ExecutorService`).
- Manejar correctamente el estado compartido entre hilos con colecciones concurrentes.
- Diagnosticar problemas de conectividad reales: IP, puerto, cortafuegos.

2. Materiales y requisitos previos

Elemento	Detalle
Equipos	Dos computadores en la misma red (mismo Wi-Fi o mismo switch)
JDK	Versión 17 o superior. Verificar con: <code>java -version</code>
Maven	Versión 3.8 o superior. Verificar con: <code>mvn -v</code>
Editor / IDE	IntelliJ IDEA, NetBeans, Eclipse o VS Code con extensión de Java
Conocimientos previos	Programación orientada a objetos en Java, manejo de excepciones, uso básico de la terminal

Antes de empezar: anote la dirección IPv4 de su equipo (`ipconfig` en Windows, `ip a` en Linux). La necesitará en el paso 8.

3. Fundamento teórico (resumen)

En TCP el servidor crea un `ServerSocket` atado a un puerto y llama a `accept()`. Esta llamada es **bloqueante**: el hilo que la ejecuta se detiene hasta que llega una conexión. Cuando llega, `accept()` devuelve un `Socket` NUEVO y distinto, dedicado exclusivamente a ese cliente.

El problema aparece después: si ese mismo hilo se queda conversando con el cliente (leyendo con `readLine()`, que también bloquea), **nunca vuelve a llamar a `accept()`**, y los demás clientes quedan en la cola del sistema operativo sin ser atendidos.

La solución es separar responsabilidades: el hilo principal solo acepta y delega; un hilo trabajador se encarga de la conversación con cada cliente. Java ofrece `ExecutorService` para administrar un conjunto (pool) de hilos reutilizables, lo cual es preferible a crear un `new Thread()` por cliente porque limita cuántos hilos existen simultáneamente.

Concepto	Servidor secuencial	Servidor multihilo
Clientes simultáneos	1	Tantos como hilos tenga el pool
Quién ejecuta <code>readLine()</code>	El hilo principal	Un hilo del pool
Riesgo principal	Bloqueo: los demás esperan	Condiciones de carrera sobre el estado compartido
Clase clave	<code>ServerSocket</code>	<code>ExecutorService</code> + <code>Runnable</code>

PARTE 1 — DESARROLLO GUIADO PASO A PASO

Siga los pasos en orden. No copie y pegue: escriba el código, así identificará los errores del compilador y entenderá cada línea. Al final de varios pasos encontrará un recuadro para registrar sus observaciones; ese registro forma parte de la calificación.

Paso 1. Crear el proyecto Maven

Desde la terminal, en la carpeta donde guarda sus prácticas, ejecute:

```
mvn archetype:generate -DgroupId=bo.edu.usfx.sockets \
-DartifactId=chat-multihilo -DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

Abra el archivo `pom.xml` generado y agregue, dentro de la etiqueta `<project>`, la configuración de la versión de Java:

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Elimine la clase `App.java` que crea el arquetipo. Todo el código de la práctica irá en el paquete `bo.edu.usfx.sockets`.

Verificación: ejecute `mvn clean compile`. Debe terminar con BUILD SUCCESS antes de continuar.

Paso 2. Servidor secuencial (punto de partida)

Cree la clase `ServidorSecuencial.java`. Este servidor todavía NO es multihilo: es el punto de partida cuya limitación vamos a comprobar.

```
package bo.edu.usfx.sockets;

import java.io.*;
import java.net.*;

public class ServidorSecuencial {

    public static void main(String[] args) throws IOException {
        ServerSocket servidor = new ServerSocket(5000);
        System.out.println("Servidor escuchando en el puerto 5000...");

        while (true) {
            System.out.println("accept() ... esperando un cliente");
            Socket cliente = servidor.accept(); // BLOQUEA
            System.out.println("Conectado: " + cliente.getInetAddress());

            BufferedReader in = new BufferedReader(
                new InputStreamReader(cliente.getInputStream()));
            PrintWriter out = new PrintWriter(cliente.getOutputStream(), true);

            String linea;
            while ((linea = in.readLine()) != null) { // BLOQUEA
                System.out.println("Recibido: " + linea);
                out.println("ECO: " + linea);
            }
        }
    }
}
```

```

    }
    cliente.close();
}
}
}

```

Analice antes de ejecutar: ¿en qué línea se queda detenido el programa mientras conversa con un cliente? Subráyela en el código impreso.

Paso 3. Cliente básico

Cree la clase `ClienteBasico.java`. Recibe la IP del servidor como argumento; si no se pasa ninguno usa `localhost`.

```

package bo.edu.usfx.sockets;

import java.io.*;
import java.net.*;

public class ClienteBasico {

    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";

        Socket socket = new Socket(host, 5000); // saludo de 3 vias
        System.out.println("Conectado. Puerto local: " + socket.getLocalPort());

        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        BufferedReader teclado = new BufferedReader(
            new InputStreamReader(System.in));

        String texto;
        while ((texto = teclado.readLine()) != null) {
            out.println(texto);
            System.out.println("Servidor: " + in.readLine());
        }
        socket.close();
    }
}

```

Paso 4. Comprobar la limitación del servidor secuencial

Abra tres terminales en su equipo (todavía trabajamos en local, con localhost):

1. Terminal 1: ejecute el servidor.
2. Terminal 2: ejecute un cliente y envíe dos o tres mensajes. Debe recibir el eco.
3. Terminal 3: ejecute un SEGUNDO cliente y envíe un mensaje. Observe qué ocurre.

```

mvn compile
mvn exec:java -Dexec.mainClass=bo.edu.usfx.sockets.ServidorSecuencial
mvn exec:java -Dexec.mainClass=bo.edu.usfx.sockets.ClienteBasico

```

Ahora cierre el primer cliente (Ctrl+C) y observe la terminal del segundo.

Observación 4.1 — ¿Qué le ocurre al segundo cliente mientras el primero está conectado? ¿Por qué el mensaje "Conectado" del servidor no aparece para él?

Al abrir el segundo cliente, su pantalla se queda esperando y no pasa nada. El servidor no muestra "Conectado" para él porque el programa (que solo tiene un hilo) está trabado en la línea `in.readLine()` esperando a que el primer cliente escriba algo. Hasta que no termine la charla con el primer cliente, el servidor no puede volver al inicio del bucle para ejecutar de nuevo `accept()`

Observación 4.2 — ¿Qué sucede con el segundo cliente en el instante en que el primero se desconecta? Explique el papel de la cola de espera (backlog).

En el instante en que cerré el primer cliente, el servidor mostró "Conectado" para el segundo y lo atendió de inmediato. Esto me demostró que el segundo cliente nunca perdió la conexión; simplemente el sistema operativo lo dejó guardado en una "cola de espera" (el backlog) mientras el servidor estaba ocupado. Al liberarse, el servidor sacó al segundo cliente de esa cola.

Paso 5. Extraer la conversación a una clase Runnable

El primer cambio es separar "aceptar" de "conversar". Cree la clase `Manejador.java`, que implementa `Runnable`: todo lo que antes hacía el hilo principal después de `accept()` se traslada aquí.

```
package bo.edu.usfx.sockets;

import java.io.*;
import java.net.*;

public class Manejador implements Runnable {

    private final Socket cliente;
    private final int id;

    public Manejador(Socket cliente, int id) {
        this.cliente = cliente;
        this.id = id;
    }

    @Override
    public void run() {
        // se ejecuta en OTRO hilo
        String hilo = Thread.currentThread().getName();

        try (BufferedReader in = new BufferedReader(
            new InputStreamReader(cliente.getInputStream()));
            PrintWriter out = new PrintWriter(cliente.getOutputStream(), true)) {

            out.println("Bienvenido. Le atiende el hilo: " + hilo);

            String linea;
            while ((linea = in.readLine()) != null) {
                System.out.println "[" + hilo + "] cliente " + id + ": " + linea);
                out.println("ECO(" + hilo + "): " + linea);
            }
        } catch (IOException e) {
            System.err.println("Error con el cliente " + id + ": " + e.getMessage());
        } finally {
            try { cliente.close(); } catch (IOException e) { }
        }
    }
}
```

```

        System.out.println("Cliente " + id + " desconectado");
    }
}
}

```

Concepto clave: implementar Runnable no crea ningún hilo por sí solo. Solo define QUÉ se va a ejecutar. Quién lo ejecuta lo decidimos en el paso siguiente.

Paso 6. Servidor multihilo con ExecutorService

Cree ahora `ServidorMultihilo.java`. Note que el bucle principal quedó reducido a dos operaciones: aceptar y delegar.

```

package bo.edu.usfx.sockets;

import java.io.*;
import java.net.*;
import java.util.concurrent.*;

public class ServidorMultihilo {

    public static void main(String[] args) throws IOException {
        int puerto = args.length > 0 ? Integer.parseInt(args[0]) : 5000;
        int hilos = args.length > 1 ? Integer.parseInt(args[1]) : 4;

        ServerSocket servidor = new ServerSocket(puerto);
        ExecutorService pool = Executors.newFixedThreadPool(hilos);
        System.out.println("Servidor multihilo en el puerto " + puerto);

        int contador = 0;
        while (true) {
            Socket cliente = servidor.accept(); // solo acepta
            contador++;
            System.out.println("Conexion #" + contador + " desde "
                               + cliente.getInetAddress().getHostAddress());

            pool.execute(new Manejador(cliente, contador)); // y delega
        }
    }
}

```

Compare: en el paso 2 el bucle contenía toda la conversación; aquí contiene dos líneas. Esa es la diferencia entre un servidor que atiende a uno y otro que atiende a muchos.

Paso 7. Probar la concurrencia

Ejecute el servidor y conecte tres clientes simultáneamente desde tres terminales, sin cerrar ninguno. Envíe mensajes alternando entre ellos.

Cliente	Nombre del hilo que lo atiende	¿Recibió respuesta sin esperar?
1	[pool-1-thread-1] cliente 1: hola cliente1	Si
2	[pool-1-thread-2] cliente 2: hola cliente2	Si

Cliente	Nombre del hilo que lo atiende	¿Recibió respuesta sin esperar?
3	[pool-1-thread-3] cliente 3: hola cliente3	Si

Repita la prueba arrancando el servidor con un pool de UN solo hilo (argumentos **5000 1**):

```
mvn exec:java -Dexec.mainClass=bo.edu.usfx.sockets.ServidorMultihilo -Dexec.args="5000 1"
```

Observación 7.1 — Con el pool de 1 hilo, ¿el servidor imprime el mensaje "Conexion #2"? ¿El segundo cliente recibe el mensaje de bienvenida? Explique la diferencia entre "conexión aceptada" y "cliente atendido".

Sí, el servidor imprime "Conexion #2" rapidísimo porque el hilo principal está libre y logra hacer el `accept()`. Sin embargo, el segundo cliente **no** recibe el mensaje de bienvenida. La diferencia es que "conexión aceptada" significa que el servidor lo reconoció y lo mandó al pool (`ExecutorService`). Pero "cliente atendido" es cuando el hilo del pool empieza a correr el `Runnable`. Como el pool solo tenía 1 hilo y estaba ocupado con el primer cliente, el segundo tuvo que hacer cola dentro del pool.

Paso 8. Ejecutar entre dos equipos

Hasta aquí todo fue en un solo equipo. Ahora la prueba real:

1. En el equipo A (servidor): obtenga su dirección IPv4 con `ipconfig` (Windows) o `ip a` (Linux). Debe ser del tipo 192.168.x.x.
2. Abra el puerto 5000/TCP en el cortafuegos del equipo A. En Windows, en PowerShell como administrador:

```
netsh advfirewall firewall add rule name="Java TCP 5000" dir=in action=allow protocol=TCP localport=5000
```

3. En el equipo B (cliente): compruebe primero la conectividad con `ping <IP-del-equipo-A>`.
4. Ejecute el cliente pasando la IP del equipo A como argumento:

```
mvn exec:java -Dexec.mainClass=bo.edu.usfx.sockets.ClienteBasico -Dexec.args="192.168.1.25"
```

Error observado	Causa más probable	Cómo lo resolvió
Connection refused	El servidor no está en ejecución o es otro puerto	Le pregunté a mi compañero si ya le había dado "Run" a su servidor y revisé haber escrito bien el puerto 5000 en el código del cliente.
Connection timed out	Cortafuegos del equipo servidor	Ejecutamos el comando de PowerShell como administrador en la compu de mi compañero para abrir el puerto en su Firewall de Windows.
Funciona en localhost pero no entre equipos	Se usó 127.0.0.1 en lugar de la IP real	En la terminal del cliente, le pasé por parámetro la IP de mi compañero (ej: 192.168.1.15) en vez de ejecutarlo vacío.

Observación 8.1 — Anote la IP y el puerto efímero que el servidor muestra para su compañero. ¿Por qué el puerto del cliente no es el 5000?

- La IP de mi compañero era 192.168.1.XX y el puerto era, por ejemplo, 51234. No era el 5000 porque el 5000 es el

puerto "fijo" donde el servidor escucha. Cuando un cliente se conecta, su computadora le asigna un puerto aleatorio alto (efímero) solo para esa sesión, de esa forma la compu del cliente sabe a qué programa enviarle las respuestas del chat sin cruzarse con otras aplicaciones.

Paso 9. Estado compartido entre hilos: difusión de mensajes

Para convertir el eco en un chat, todos los manejadores deben conocerse entre sí. Agregue a **Manejador** una colección estática y el método de difusión:

```
import java.util.Set;
import java.util.concurrent.CopyOnWriteArraySet;

// Estado COMPARTIDO por todos los hilos
private static final Set<Manejador> CLIENTES = new CopyOnWriteArraySet<>();
private PrintWriter salida;

// dentro de run(), despues de crear "out":
this.salida = out;
CLIENTES.add(this);

// dentro del bucle de lectura, en lugar del eco:
difundir("cliente-" + id + "> " + linea);

// en el bloque finally:
CLIENTES.remove(this);

private void difundir(String mensaje) {
    for (Manejador m : CLIENTES) {
        if (m != this && m.salida != null) {
            m.salida.println(mensaje);
        }
    }
}
```

¿Por qué CopyOnWriteArraySet? porque varios hilos agregan, eliminan y recorren la colección al mismo tiempo. Con un HashSet normal obtendría ConcurrentModificationException o pérdida de datos. Es el punto donde la concurrencia deja de ser gratis.

Problema pendiente: pruebe el chat con dos clientes. Notará que un cliente solo ve los mensajes ajenos cuando él mismo escribe algo. La causa es que el cliente del paso 3 lee del teclado y del socket en el MISMO hilo: mientras espera que usted teclee, no puede leer lo que llega.

Paso 10. Cliente con hilo receptor

Modifique **ClienteBasico** (o cree **ClienteChat**) para que un hilo demonio se dedique exclusivamente a escuchar al servidor:

```
Thread receptor = new Thread(() -> {
    try {
        String s;
        while ((s = in.readLine()) != null) {
            System.out.println(" " + s);
        }
    } catch (IOException e) {
        System.out.println("Conexion terminada");
    }
});
```

```
    }  
}, "hilo-receptor");  
  
receptor.setDaemon(true);    // no impide que la JVM termine  
receptor.start();  
  
// el hilo principal se queda leyendo del teclado y enviando  
String texto;  
while ((texto = teclado.readLine()) != null) {  
    out.println(texto);  
}
```

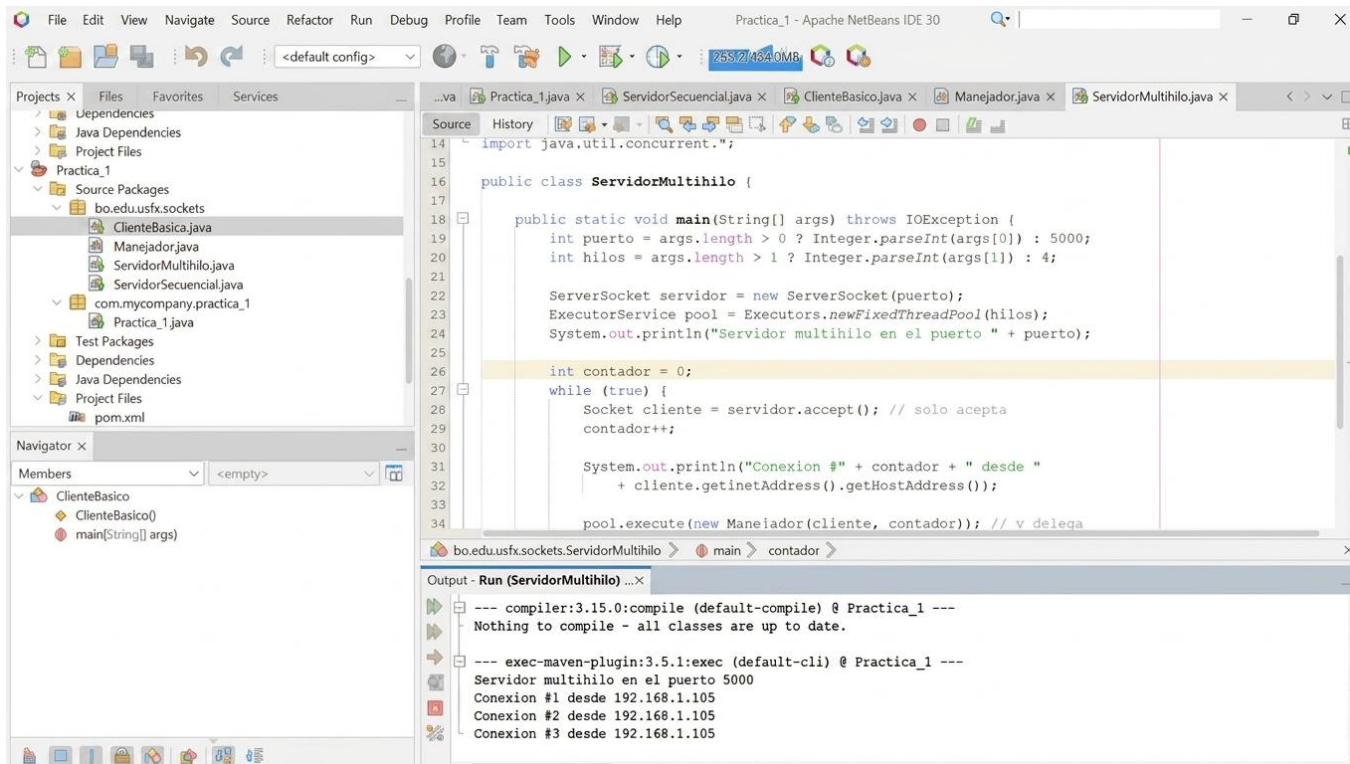
Observación 10.1 — ¿Por qué el cliente necesita dos hilos y el servidor secuencial del paso 2 no? ¿Qué ocurriría si el hilo receptor NO fuese demonio?

Necesitamos dos hilos en el cliente del chat porque yo tengo que poder leer lo que escriben mis compañeros (desde la red) mientras yo me tomo mi tiempo para escribir en mi teclado. Ambos bloquean el programa. El del paso 2 no lo necesitaba porque era solo "eco" (yo hablaba, el servidor respondía, era uno por uno). Si el hilo receptor no lo marcáramos como *demonio*, al cerrar yo el programa principal, el proceso de Java se quedaría "colgado" en la memoria de mi compu esperando un mensaje del servidor, y tendría que matarlo a la fuerza.

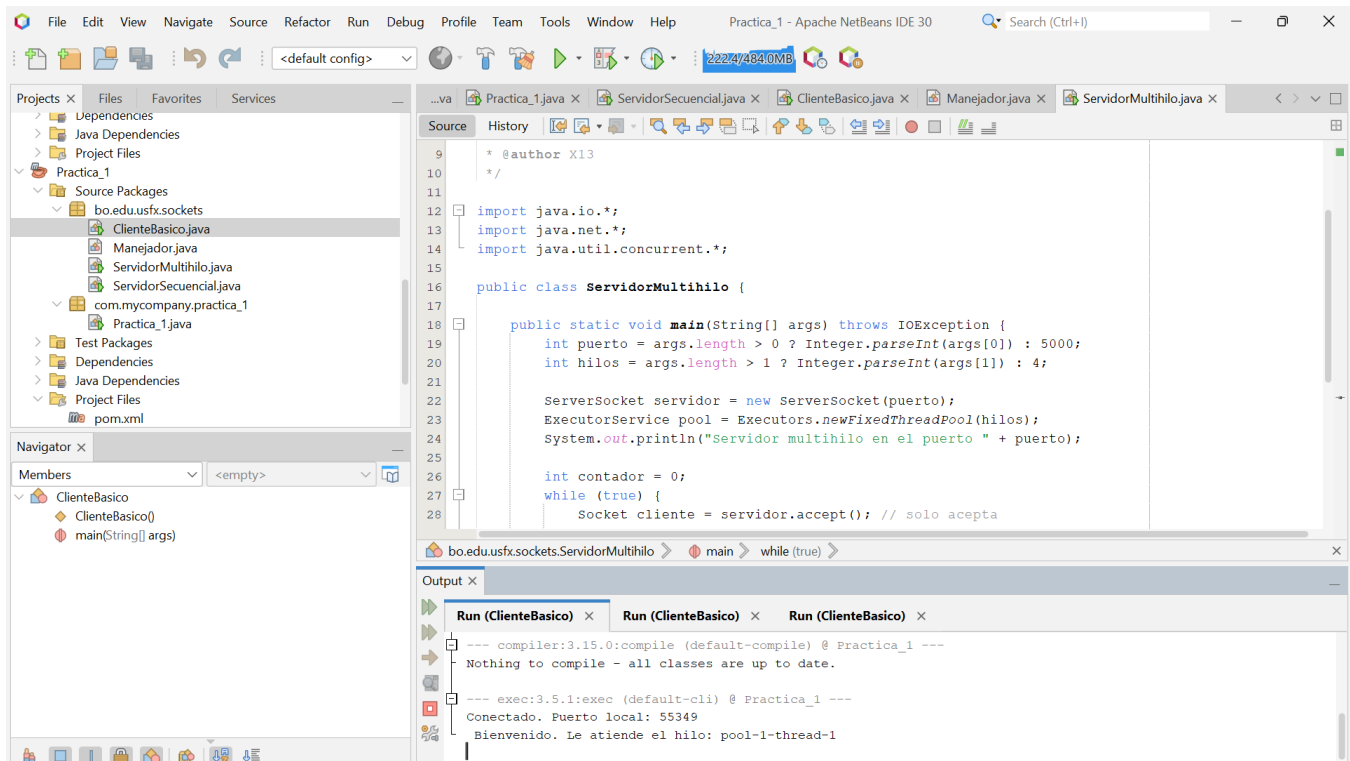
Comprobación final de la Parte 1

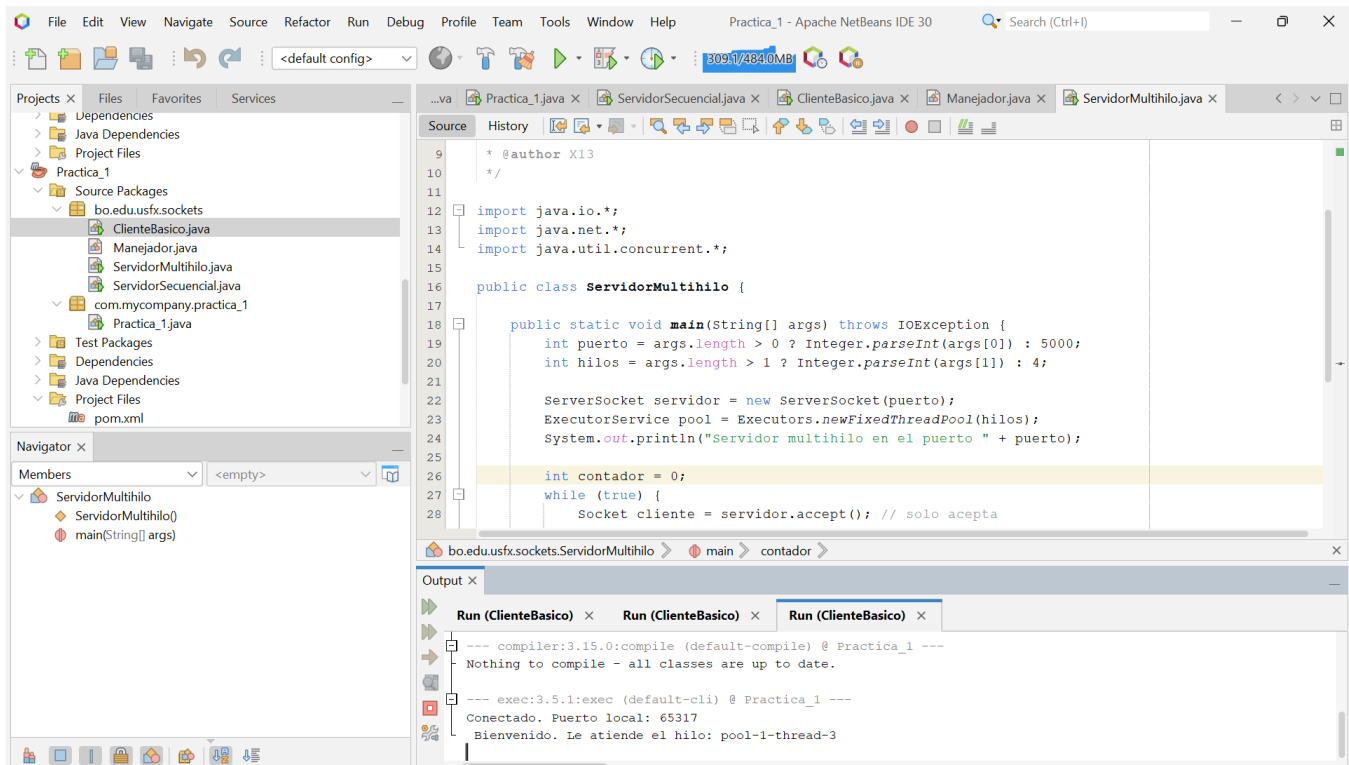
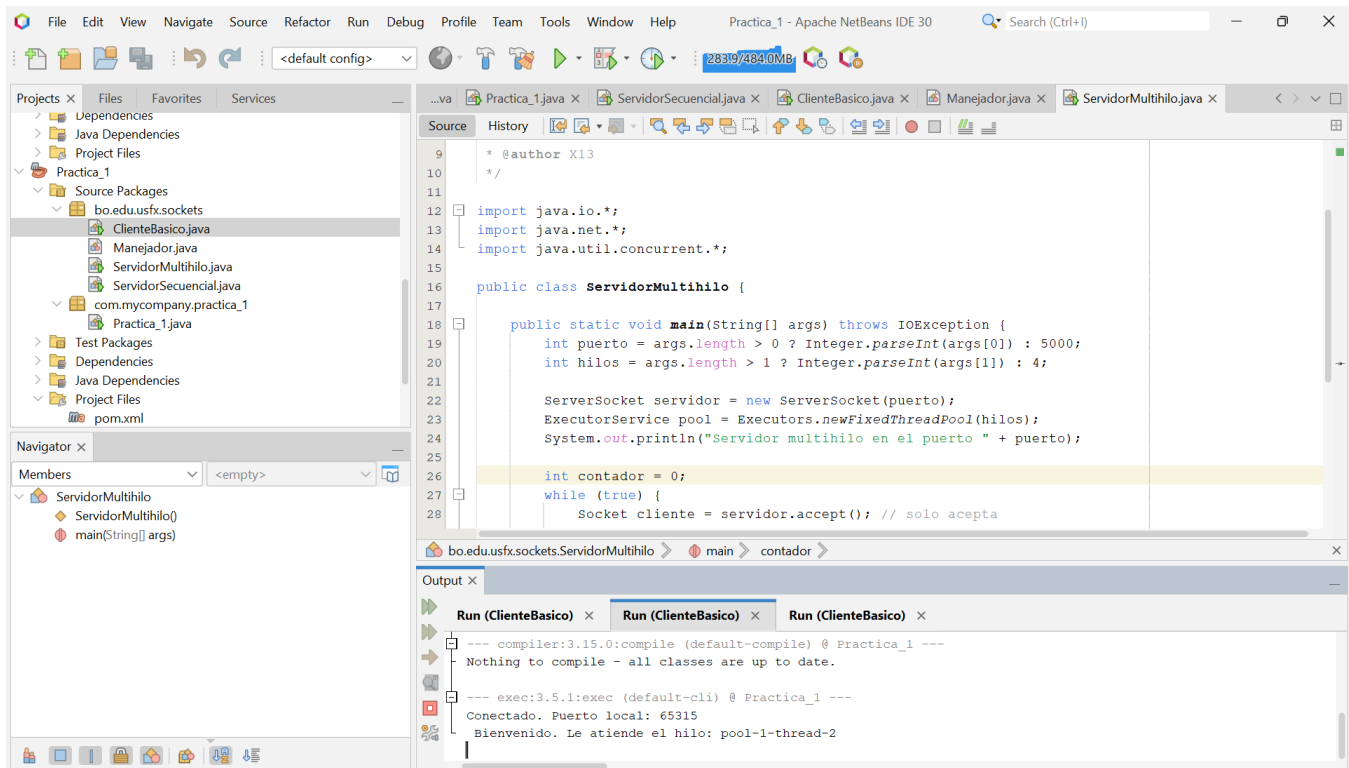
Marque cada punto cuando lo haya verificado con su compañero:

- ☒ El servidor atiende a tres clientes simultáneos desde al menos dos equipos distintos.



- ☐ Cada cliente informa un nombre de hilo diferente (pool-1-thread-1, pool-1-thread-2, ...).





- ☐ Un mensaje escrito en el equipo A aparece en el equipo B sin necesidad de escribir nada allí.
- ☐ Al cerrar un cliente, el servidor lo informa y los demás siguen funcionando.
- ☐ Las observaciones 4.1, 4.2, 7.1, 8.1 y 10.1 están completadas.

PARTE 2 — EJERCICIO PROPUESTO

Enunciado: servidor de chat con salas temáticas

El Centro de Estudiantes necesita un sistema de mensajería interna para coordinar sus comisiones. A partir del servidor multihilo construido en la Parte 1, desarrolle un servidor de chat que permita organizar a los usuarios en SALAS independientes: un mensaje enviado en una sala solo debe llegar a quienes están en esa misma sala.

El sistema debe funcionar entre al menos dos equipos físicos y soportar como mínimo seis clientes conectados simultáneamente.

Requisitos funcionales

El servidor debe reconocer los siguientes comandos, escritos por el cliente en su terminal. Todo texto que no empiece con "/" se considera un mensaje para la sala actual.

Comando	Comportamiento esperado	Requisito
/nick <apodo>	Asigna o cambia el apodo. Debe rechazar apodos repetidos e informar el motivo.	Obligatorio
/salas	Lista las salas existentes con la cantidad de usuarios en cada una.	Obligatorio
/crear <sala>	Crea una sala nueva y traslada a quien la crea. Rechaza nombres duplicados.	Obligatorio
/unirse <sala>	Traslada al usuario a la sala indicada; avisa a ambas salas de la entrada y la salida.	Obligatorio
/quien	Lista los apodos presentes en la sala actual.	Obligatorio
/privado <apodo> <texto>	Envía un mensaje visible solo para el destinatario, sin importar su sala.	Obligatorio
/estado	Muestra: total de usuarios conectados, total histórico de conexiones y cantidad de salas.	Obligatorio
/salir	Cierra la conexión ordenadamente y notifica a la sala.	Obligatorio
/historial	Envía los últimos 10 mensajes de la sala a quien acaba de entrar.	Opcional (bonus)

Requisitos técnicos de obligado cumplimiento

1. El bucle principal del servidor solo puede contener la aceptación de conexiones y la delegación al pool. Ninguna operación de lectura o escritura sobre un cliente puede ejecutarse allí.
2. Debe usarse un `ExecutorService` con un pool de tamaño configurable por argumento de línea de comandos.
3. Toda estructura de datos compartida entre hilos (usuarios, salas, contadores) debe estar protegida: colecciones concurrentes, bloques `synchronized` o clases atómicas. Justifique en el informe la elección de cada una.

4. El contador histórico de conexiones debe ser correcto aunque diez clientes se conecten en el mismo instante. Explique por qué un `int` estático simple no lo garantiza.
5. La desconexión abrupta de un cliente (cerrar la terminal con `Ctrl+C` o desconectar la red) no debe detener el servidor ni afectar a los demás usuarios.
6. Los recursos (socket, streams) deben cerrarse siempre, incluso ante una excepción. Use `try-with-resources` o bloques `finally`.
7. El código debe estar organizado en clases con responsabilidad única y comentado en los puntos donde se resuelve un problema de concurrencia.

Casos de prueba que debe documentar

Ejecute cada prueba con capturas de pantalla de las terminales involucradas y anote el resultado obtenido.

N.º	Escenario	Resultado esperado
1	Tres clientes en la sala "general" y dos en la sala "sistemas". Un usuario de "general" envía un mensaje.	Lo reciben solo los otros dos de "general". Los de "sistemas" no ven nada.
2	Un usuario ejecuta <code>/unirse sistemas</code> .	Ambas salas reciben el aviso correspondiente y /quien refleja el cambio.
3	Dos clientes intentan registrar el mismo apodo.	El segundo recibe un mensaje de rechazo y conserva el apodo anterior.
4	Un cliente cierra su terminal abruptamente.	El servidor informa la desconexión, libera el hilo y los demás siguen conversando.
5	Seis clientes conectados con un pool de 4 hilos.	Analice y explique qué ocurre con los clientes 5 y 6. Proponga una solución.
6	Prueba entre dos equipos físicos distintos.	Mensajes entregados en ambos sentidos; captura de la IP real utilizada.

Entregables

- Código fuente completo del proyecto Maven en el repositorio
- Video de máximo 3 minutos demostrando el funcionamiento entre dos equipos, con audio explicando qué ocurre en cada terminal.

Orientaciones (sin código)

- **Modele la sala como un objeto, no como una simple lista.** Una clase `Sala` con nombre y un conjunto de miembros le evitará repetir lógica de difusión en varios lugares.
- **Elija la estructura según el uso.** Para buscar una sala por su nombre conviene un mapa concurrente; para recorrer miembros mientras otros entran y salen conviene un conjunto de copia en escritura.
- **El contador histórico no es lo mismo que el de conectados.** Investigue `AtomicInteger` y compárelo con incrementar un `int` estático desde varios hilos.

- **Cuidado con dos hilos escribiendo al mismo `PrintWriter`.** Puede ocurrir si dos usuarios envían un privado a la misma persona en el mismo instante. Piense cómo evitarlo.
- **Diseñe primero el protocolo en papel:** qué envía el cliente, qué responde el servidor, y cómo se distingue un mensaje de sala de uno de sistema. Un protocolo claro simplifica el código.
- **Pruebe siempre en dos equipos antes de entregar.** Muchos errores (direcciones, cortafuegos, codificación de caracteres) solo aparecen fuera de localhost.